{EPITECH}

CONNECT'IN V2_

< MIGRATION DU BACK-END VERS JAVA SPRING BOOT />



CONNECT'IN V2

Contexte professionnel

Vous avez terminé la première version de **Connect'In**, le réseau social interne de l'entreprise. Vos collaborateurs en sont très satisfaits, mais la direction technique souhaite faire évoluer la solution.

Même si la première version développée avec Laravel fonctionne correctement, l'entreprise souhaite désormais harmoniser ses technologies back-end autour de **Java** afin de faciliter la maintenance, améliorer la robustesse globale des applications et s'aligner sur les standards techniques utilisés dans ses autres projets.

Dans ce contexte, votre équipe reçoit la mission de **remplacer entièrement le back-end existant** par une nouvelle version développée avec **Spring Boot**, tout en conservant le front-end déjà en place. Cette contrainte implique que la nouvelle API devra rester compatible avec l'application cliente existante, sans modification de celle-ci.

Vous êtes donc chargé de concevoir et développer **Connect'In v2**, une nouvelle API reposant sur l'écosystème Java.

Présentation du projet

Le projet consiste à recréer le back-end de l'application sous forme d'une **API RESTful Spring Boot**, en reproduisant fidèlement le comportement de l'API précédente.



L'équipe pédagogique locale sélectionnera les deux meilleurs front-end réalisés lors de la première édition de **Connect'In**. Ces deux projets serviront de base commune pour l'ensemble des groupes. Chaque groupe sera ensuite affecté à l'un de ces front-end, en veillant à ce qu'aucun participant ne travaille sur celui qu'il a lui-même développé.
Le front-end ayant déjà été conçu et validé, il ne devra faire l'objet **d'aucune modification**. Cela implique notamment de conserver l'intégralité de la logique métier, les routes API existantes ainsi que les formats de réponses JSON attendus. L'objectif principal n'est donc pas d'ajouter de nouvelles fonctionnalités, mais de démontrer votre capacité à comprendre une architecture existante et à la reproduire fidèlement dans une autre technologie.

L'application reposera sur une base de données MySQL et devra suivre à nouveau une organisation professionnelle en couches séparant clairement les responsabilités (contrôleurs, services, accès aux données, sécurité, etc.).

Objectifs pédagogiques

Ce projet vise à vous confronter à une situation réaliste de migration technologique telle qu'on peut en rencontrer en entreprise. Il vous permettra notamment de :

- ✓ comprendre et analyser une API existante
- ✓ concevoir une API REST avec Spring Boot
- ✓ utiliser Spring Data JPA et Hibernate pour la persistance
- ✓ mettre en place une authentification sécurisée par token (JWT)
- ✓ structurer une application Java selon une architecture en couches
- ✓ maintenir la compatibilité entre un front-end et un back-end
- ✓ appliquer les bonnes pratiques professionnelles du développement Java
- ✓ déployer une application conteneurisée avec Docker

Au-delà de l'aspect technique, ce projet met également l'accent sur la capacité à lire, comprendre et reproduire un système existant.

Fonctionnalités attendues

Les fonctionnalités implémentées devront être équivalentes à celles de la première version du projet Connect'In. Pour rappel, les fonctionnalités attendues sont les suivantes :

L'application devra permettre la **gestion complète des utilisateurs** : inscription, authentification, consultation et modification du profil, changement de mot de passe ainsi que suppression du compte. Lorsqu'un utilisateur supprime son compte, il devra pouvoir choisir entre supprimer l'ensemble de ses contenus ou les conserver sous forme anonymisée. Dans tous les cas, les informations personnelles et les likes associés devront être supprimés.

La **gestion des publications** constitue le cœur du réseau social. Un utilisateur connecté pourra créer un post contenant du texte et éventuellement une image, modifier ses propres publications et les supprimer. Tous les utilisateurs authentifiés pourront consulter les publications des autres collaborateurs et interagir avec celles-ci via un système de likes. Chaque utilisateur ne pourra liker un post qu'une seule fois. La suppression d'un post entraînera automatiquement la suppression des commentaires associés.

Le système devra également permettre la gestion des commentaires. Chaque publication pourra recevoir plusieurs commentaires et chaque utilisateur devra pouvoir modifier ou supprimer uniquement ses propres messages.

Comme dans la première version, toutes les interactions devront fonctionner de manière dynamique grâce aux appels API réalisés par le front-end existant. Les réponses envoyées par votre API devront donc être immédiatement exploitables sans nécessiter de rechargement de page.

Contraintes techniques

Le projet devra être réalisé avec les technologies suivantes :

- ✓ Java 25 (ou version plus récente)
- ✓ Spring Boot
- ✓ Spring Web
- ✓ Spring Data JPA
- ✓ Spring Security
- ✓ JWT pour l'authentification
- ✓ MySQL
- ✓ Maven ou Gradle

Conteneurisation (Docker)

Le projet devra être entièrement dockerisé.

Cela implique :
- un Dockerfile pour l'application Spring Boot
- un docker-compose.yml permettant de lancer :
- l'application
- la base de données MySQL
- la configuration des variables d'environnement (BDD, JWT, etc.)
- un démarrage fonctionnel du projet via la commande :

```
docker compose up --build
```

L'objectif est de fournir un environnement reproductible et professionnel, similaire à un contexte réel d'entreprise.

Architecture attendue

Une **architecture claire et structurée** est attendue, avec une séparation logique entre les différentes couches de l'application (contrôleurs, services, repositories, entités, configuration, sécurité, gestion des exceptions, etc.).

La sécurité devra être correctement implémentée. Toutes les routes nécessitant une authentification devront être protégées, et les règles d'autorisation devront garantir qu'un utilisateur ne puisse modifier ou supprimer que ses propres ressources.

Base de données

Vous avez la possibilité de réutiliser le modèle de base de données existant ou de le redéfinir si nécessaire, à condition de respecter les fonctionnalités attendues et de conserver le même d'entrées/sorties de vos routes.

Un diagramme de base de données devra être fourni dans les livrables afin de justifier vos choix de conception en cas de changements.

Bonus (facultatif)

Une fois toutes les fonctionnalités principales implémentées, vous pourrez enrichir votre projet avec :

- ✓ des tests unitaires et d'intégration
- ✓ l'optimisation des images uploadées
- ✓ la mise en cache de certaines données

Livrables attendus

Les éléments suivants devront être fournis :

- ✓ le code source complet du projet
- ✓ un README détaillant l'installation, la configuration et le lancement
- ✓ un diagramme de base de données
- ✓ une documentation de l'API (PDF, collection Postman, interface web ou autre)

Ressources utiles

Framework

Spring Boot — https://spring.io/projects/spring-boot
Spring Security — https://spring.io/projects/spring-security
Spring Data JPA — https://spring.io/projects/spring-data-jpa
JWT — https://jwt.io
MySQL — https://www.mysql.com
Hibernate — https://hibernate.org
Maven — https://maven.apache.org

Tests

Spring Boot Test — https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing
JUnit 5 — https://junit.org/junit5/
Mockito — https://site.mockito.org/
Testcontainers — https://www.testcontainers.org/

Conteneurisation

Docker — https://www.docker.com/
Docker Compose — https://docs.docker.com/compose/

v 1.0.1

